

SISTEMA DE MONITOREO DE SIGNOS VITALES PEDIÁTRICO BASADO EN ATMEGA328P ATMEGA328P-BASED PEDIATRIC VITAL SIGNS MONITORING SYSTEM

Mayra Alejandra García Garrido¹, Lesly Guadalupe López Moreno²

Resumen

Los niños con enfermedades crónicas necesitan monitoreo constante de sus signos vitales, pero los dispositivos médicos convencionales los asustan y generan rechazo, lo que complica la adquisición continua de datos. Este trabajo presenta un sistema embebido de monitoreo de frecuencia cardíaca, saturación de oxígeno (SpO₂) y temperatura corporal, basado en el ATmega328P programado en C con Microchip Studio. Los sensores MAX30102 y MLX90614 se comunican mediante I2C; los datos se muestran en una LCD 20×4 y se transmiten a PC por USART. El muestreo periódico se maneja con interrupciones del Timer1, sin bloquear la ejecución del programa. Todo el sistema va integrado en un peluche con forma de conejita, con el objetivo de que el dispositivo le resulte familiar al paciente pediátrico en lugar de intimidante.

Palabras Clave: ATmega328P, signos vitales, pediátrico, MAX30102, MLX90614, sistemas embebidos.

¹ Ing. en Computación, #0350101 — mayra.alejandra.garcia.garrido@uabc.edu.mx

² Ing. en Computación, #1187211 — lesly.lopez19@uabc.edu.mx

Abstract

Children with chronic conditions require continuous vital sign monitoring, but conventional medical devices tend to frighten young patients and generate resistance, making data acquisition difficult. This paper presents an embedded monitoring system for heart rate, oxygen saturation (SpO₂), and body temperature, built around the ATmega328P microcontroller programmed in C using Microchip Studio. The MAX30102 and MLX90614 sensors communicate via I2C; data are displayed on a 20×4 LCD and transmitted to a PC through USART. Periodic sampling is handled by Timer1 interrupts, keeping the program execution non-blocking. The entire system is housed in a bunny stuffed toy enclosure, designed to feel familiar rather than threatening to pediatric patients.

Keywords: ATmega328P, vital signs, pediatric, MAX30102, MLX90614, embedded systems.

1. Introducción

El monitoreo de signos vitales en niños con enfermedades crónicas o condiciones que requieren vigilancia constante es una necesidad clínica real. Cambios en la frecuencia cardíaca, la saturación de oxígeno o la temperatura pueden aparecer días antes de que la situación se vuelva crítica [1], y detectarlos a tiempo puede marcar la diferencia entre una intervención oportuna y una hospitalización de emergencia. Sin embargo, lograr ese monitoreo en pacientes pediátricos no es tan sencillo como colocar un sensor; el dispositivo también tiene que ser algo que el niño acepte usar.

1.1 Contexto

Los dispositivos médicos convencionales fueron diseñados pensando en adultos. Pinzas en los dedos, electrodos, termómetros de contacto son incómodos incluso para un paciente adulto, y para un niño pequeño pueden ser motivo de llanto, resistencia activa o simplemente negativa a quedarse quieto durante la medición. Este fenómeno está bien documentado; la ansiedad que generan los entornos y equipos médicos afecta directamente la calidad de los datos obtenidos y puede retrasar decisiones clínicas importantes [2, 3].

Al mismo tiempo, la tecnología disponible para monitoreo portátil ha avanzado considerablemente. Sensores como el MAX30102 (capaz de medir frecuencia cardíaca y saturación de oxígeno mediante fotoplethysmografía), y el MLX90614 (termómetro infrarrojo sin contacto) ofrecen mediciones precisas en formatos compactos y de bajo costo, con comuni-

cación I2C que los hace compatibles con microcontroladores como el ATmega328P [6, 7]. Esto abre la posibilidad de construir sistemas de monitoreo clínicamente útiles que no dependan de infraestructura hospitalaria costosa.

1.2 Descripción del problema

El problema central es que existe una brecha entre lo que los dispositivos médicos pueden medir y lo que los pacientes pediátricos están dispuestos a tolerar. Un niño que rechaza el sensor produce datos incorrectos o simplemente no produce datos. En contextos donde el monitoreo debe ser continuo (como en niños con enfermedades crónicas que requieren seguimiento en casa) esta resistencia al equipo médico se convierte en un obstáculo [1, 3].

Los cuidadores de niños con condiciones médicas complejas han señalado que preferirían monitorear a sus hijos en casa si existiera una herramienta accesible y fácil de usar [1]. Sin embargo, la mayoría de los dispositivos disponibles comercialmente siguen siendo voluminosos, costosos o visualmente intimidantes para la población pediátrica. No existe una solución que combine medición confiable de los parámetros más relevantes (saturación de oxígeno, frecuencia cardíaca y temperatura) con un diseño que resulte familiar y no amenazante para un niño.

1.3 Soluciones previas

Varios grupos han intentado abordar este problema desde distintos ángulos. Vonach et al. desarrollaron “MediCubes”,

un sistema donde sensores de pulso, temperatura y saturación de oxígeno van integrados en bloques de madera que forman parte de un juego de cuentos. En un estudio con niños de 5 a 12 años, más del 60 % calificó la experiencia con la puntuación más alta en la escala de disfrute, y los participantes adultos señalaron que los cubos no les recordaban a equipo médico [2]. El trabajo demuestra que el diseño del dispositivo tiene un impacto real en la aceptación del paciente.

Prihatini et al. tomaron una dirección similar con un robot socialmente asistivo para niños con autismo, equipado con el MAX30102 y el MLX90614. Sus resultados mostraron que la forma del dispositivo influye directamente en cuánto coopera el niño durante la medición, y que los signos vitales registrados correlacionan con los niveles de ansiedad del paciente [4]. Nkoy et al. consultaron directamente a cuidadores de niños con condiciones médicas complejas sobre qué querían monitorear en casa; la respuesta fue saturación de oxígeno, frecuencia cardíaca y temperatura, exactamente los parámetros que cubre este proyecto [1].

En cuanto a la implementación técnica, Bejarano-Reyes et al. construyeron un sistema de monitoreo remoto con el MAX30102 y el MLX90614 sobre una plataforma Arduino y lo compararon estadísticamente con equipo comercial, sin encontrar diferencias significativas en las mediciones [5]. Khalafullah et al. usaron específicamente el ATmega328P en un sistema portátil de monitoreo probado en pacientes de distintas edades, con resultados de alta precisión para frecuencia cardíaca y temperatura [6].

Lo que ninguno de estos trabajos combina es una implementación con un diseño orientado específicamente a reducir el rechazo del paciente pediátrico mediante una carcasa con apariencia de personaje animado conocido.

1.4 Objetivos

El objetivo general es diseñar e implementar un sistema de monitoreo de signos vitales programado en C sobre un ATmega328P en Microchip Studio, dentro de un contenedor con apariencia amigable, orientado a pacientes pediátricos.

Los objetivos específicos son:

- Implementar un driver de comunicación I2C en C para leer el MAX30102 y el MLX90614.
- Desarrollar un driver para pantalla LCD 20×4 para mostrar los datos en tiempo real.
- Configurar el módulo USART para enviar datos a una PC.
- Usar el Timer1 para muestreo periódico mediante interrupciones, sin bloquear la ejecución del programa.
- Integrar toda la electrónica en un contenedor con diseño amigable para el paciente pediátrico.

2. Materiales y Métodos

El sistema propuesto consiste en un dispositivo embebido que adquiere datos de dos sensores mediante el bus I2C del ATmega328P, los procesa en tiempo real y los presenta tanto en una pantalla LCD local como en una interfaz serial hacia PC. Todo el firmware se desarrolla en lenguaje C en Micro-

chip Studio, sin dependencias de bibliotecas externas.

2.1 Periféricos

Se utilizan cuatro periféricos del ATmega328P, conforme a los requerimientos del proyecto:

1. **I2C (TWI):** Comunicación con el MAX30102 y el MLX90614 [8, 9]. El bus opera a 100 kHz en modo estándar.
2. **USART:** Transmisión de datos a 9600 baud hacia una computadora para visualización en monitor serial.
3. **Timer1:** Configurado en modo CTC para generar una interrupción cada 100 ms, que establece una bandera de muestreo. El *loop* principal revisa esta bandera para ejecutar la lectura de sensores sin bloquear la ejecución.
4. **LCD 20×4 (HD44780):** Interfaz de visualización local. Se controla en modo de 4 bits mediante el expansor de puertos PCF8574 integrado en el módulo de pantalla, conectado al bus I2C.

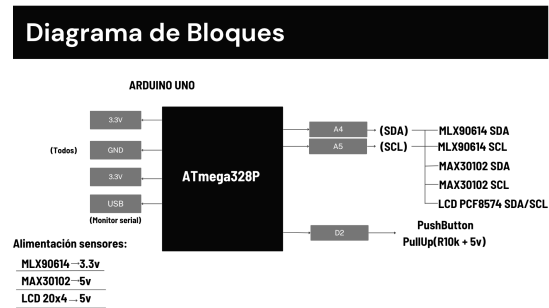


Figura 1: Diagrama de bloques del sistema de monitoreo de signos vitales pediátrico.

La Figura 1 muestra la arquitectura general del sistema. El ATmega328P actúa como unidad central de procesamiento. Los sensores MAX30102 y MLX90614 se conectan al bus TWI (pines SDA/SCL) [6, 8]. La pantalla LCD 20×4 se controla mediante el módulo I2C PCF8574 incluido en el módulo de pantalla. La comunicación con PC se realiza mediante el módulo USART (TX/RX). El muestreo es controlado por el Timer1 mediante interrupciones periódicas de 100 ms.

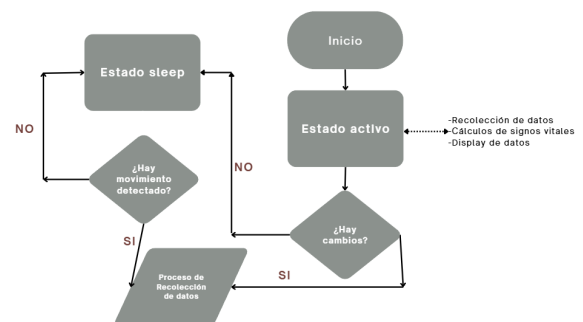


Figura 2: Diagrama de flujo

2.2 Requisitos del sistema

La Tabla 1 lista los requisitos funcionales y no funcionales que el sistema debe satisfacer.

Cuadro 1: Requisitos del sistema de monitoreo de signos vitales pediátrico.

Requisito	Especificación
Medir frecuencia cardíaca (HR)	MAX30102 vía I2C; rango esperado 40–200 bpm [6]
Medir saturación de oxígeno (SpO2)	MAX30102 vía I2C; precisión ±2 % [6]
Medir temperatura corporal sin contacto	MLX90614 vía I2C; rango 0–50 °C [8]
Mostrar datos en pantalla local	LCD 20×4 con todos los parámetros visibles simultáneamente
Transmitir datos a PC	USART a 9600 baud para logging y monitoreo remoto
Muestreo periódico sin bloqueo	Timer1 con ISR cada 100 ms; sin uso de delay ()
Programado en C bare-metal	Microchip Studio, sin bibliotecas de Arduino
Carcasa amigable para niños	Peluche con diseño familiar para el paciente pediátrico
Alimentación compacta	5 V USB o batería; sistema portátil
Mínimo 4 periféricos del ATmega328P	I2C, USART, Timer1, (LCD como interfaz de salida)

2.3 Presupuesto

La Tabla 2 detalla los materiales requeridos para la implementación del proyecto, con sus costos estimados.

Cuadro 2: Presupuesto del proyecto.

Componente	Costo estimado
ATmega328P (Arduino NANO)	\$122 MXN
MAX30102 (HR + SpO2)	\$85.07 MXN
MLX90614 (temperatura IR)	\$189.73 MXN
LCD 20×4 con módulo I2C	\$124 MXN
Peluche conejita con vestido floral	\$329 MXN
TOTAL	\$849.8 MXN

2.4 Planeación

El proyecto se desarrollará en cuatro semanas a partir del 28 de abril de 2025. La Tabla 3 presenta el cronograma de actividades.

3. Resultados

El sistema quedó funcionando de extremo a extremo. A continuación se describe lo que se logró, los ajustes que fue

Cuadro 3: Cronograma de actividades.

Período	Actividades	S1	S2	S3	S4
Sem. 28 Abr –4 May	Driver TWI/I2C en C puro (ATmega328P). Validación con MLX90614.	✓	✓		
Sem. 5–11 May	Driver MAX30102: configuración de registros y lectura.			✓	
Sem. 12–18 May	Driver LCD 20×4. Integración USART y Timer1 ISR. Ensamble en PCB.				✓
Sem. 19–25 May	Ensamble en carcasa. Pruebas funcionales completas. Reporte final.				✓

necesario hacer sobre la marcha, y los problemas que se resolvieron durante el desarrollo.

3.1 Implementación de los drivers

Se escribieron los cuatro módulos de software necesarios para que el microcontrolador pudiera hablar con cada componente del sistema. El módulo de comunicación I2C se programó directamente sobre los registros del ATmega328P, con un mecanismo de timeout para que el sistema no se quedara colgado si algún sensor dejaba de responder. El módulo del MAX30102 lo configura al encender y lee sus datos de manera que no se pierdan ni se mezclen muestras. El del MLX90614 obtiene la temperatura del objeto apuntado. El del LCD lo inicializa y le manda caracteres para mostrar la información en pantalla.

El Timer1 se configuró para generar una señal cada 100 ms, que es lo que marca el ritmo de todo el sistema: diez veces por segundo el programa revisa los sensores, calcula los valores y actualiza la pantalla, sin bloquear ninguna otra parte del código mientras espera. La diferencia respecto a usar un delay () es clara: en lugar de congelar el microcontrolador cien milisegundos, la interrupción simplemente activa una bandera y el programa continúa.

Listing 1: ISR del Timer1 y uso en el loop principal.

```
1 // La interrupcion solo activa una bandera
2 ISR(TIMER1_COMPA_vect) {
3     sample_flag = 1;
4 }
5
6 // El loop principal revisa la bandera y sigue su camino
7 if (sample_flag) {
8     sample_flag = 0;
9     // leer sensores, calcular, actualizar LCD...
10 }
```

3.2 Calibración: por qué fue necesaria

Las fórmulas que vienen en las hojas técnicas de los sensores no funcionaron directamente. El problema principal fue que el módulo MAX30102 que se consiguió es una versión

clon (MH-ET LIVE) que tiene los canales de luz roja e infrarroja al revés respecto al chip original. Eso significó que todos los cálculos de saturación de oxígeno y frecuencia cardíaca tuvieron que ajustarse manualmente comparando las lecturas del sistema contra un oxímetro comercial hasta que los números coincidieran. La Tabla 4 resume qué se midió, cuál era el valor real y qué corrección se aplicó.

Cuadro 4: Calibraciones validadas contra instrumentos de referencia.

Parám.	Referencia	Calibración aplicada
Temperatura	36.9 °C	Offset +3.2 °C (compensación piel → corporal)
SpO2	94–96 %	$SpO2 = 110 - 14 \cdot R$, con $R = RED/IR$
HR	73 bpm	$HR = 600/I_{prom}$ + filtro de mediana ($n = 7$)

En el caso del SpO2, la fórmula final que funcionó fue la siguiente, donde R es la razón entre la señal roja y la infrarroja:

Listing 2: Cálculo de SpO2 calibrado empíricamente.

```
1 float ratio = (float)red / (float)ir;
2 float spo2 = 110.0f - 14.0f * ratio;
3 // Ejemplo: ratio=1.0 -> SpO2=96%
4 //          ratio=1.1 -> SpO2=94.6%
```

La versión anterior usaba los coeficientes del datasheet original (123 y 23) y siempre daba 99–100 %, sin importar el dedo. Los valores actuales se ajustaron hasta que el sistema coincidió con el oxímetro de referencia.

3.3 Cómo se llegó al algoritmo de frecuencia cardíaca

Este fue el parámetro que más iteraciones requirió. La primera versión simplemente contaba cuántos latidos detectaba en cinco segundos y los multiplicaba por una constante. El resultado era muy impreciso: los valores solo podían salir en ciertos saltos fijos (48, 64, 80, 96...) y nunca daban un número intermedio.

La solución fue cambiar el enfoque: en lugar de contar picos, se mide cuánto tiempo pasa entre un latido y el siguiente, se promedia ese intervalo y se convierte a pulsaciones por minuto. Con eso los valores ya podían ser cualquier número continuo. Aun así, de vez en cuando salía una lectura disparada por ruido en la señal. Para filtrar esos errores se guardan los últimos siete resultados y se reporta el valor del centro, no el promedio, porque el promedio se contamina con un dato malo pero el valor central simplemente lo ignora.

La Tabla 5 muestra cómo fue evolucionando el algoritmo.

El fragmento de código que implementa el cálculo final muestra bien por qué el 600 de la fórmula viene directamente de multiplicar 60 segundos por las 10 muestras por segundo del Timer1.

Listing 3: Cálculo de HR por intervalo entre picos (v4 final).

Cuadro 5: Evolución del algoritmo de cálculo de HR.

Ver.	Método	Problema
v1	$HR = picos \cdot 12$	Salto de 12 bpm
v2	$HR = picos \cdot 16$	Salto de 16 bpm
v3	$HR = \frac{600}{I_{prom}}$	Variación 40–120 bpm
v4	v3 + mediana ($n = 7$)	Estable, ± 5 bpm

```
1 // Intervalo promedio entre picos consecutivos
2 uint16_t total = 0;
3 for (i = 1; i < num_peaks; i++)
4     total += peak_pos[i] - peak_pos[i-1];
5 float avg_interval = (float)total / (num_peaks - 1);
6
7 // 600 = 60 s x 10 muestras/s
8 uint8_t hr = (uint8_t)(600.0f / avg_interval);
9
10 // Mediana de 7: el centro descarta lecturas malas
11 hr = median_hr(hr);
```

3.4 Compensación de temperatura

El sensor de temperatura mide la superficie de la piel, no la temperatura interna del cuerpo. En el dedo esa diferencia es de unos 2.5 a 3.5 °C, dependiendo de la persona y las condiciones. Al comparar la lectura del sensor (33.7 °C) contra un termómetro de referencia (36.9 °C) se determinó que había que sumarle 3.2 °C a todo lo que reportara el sensor para que el valor mostrado fuera clínicamente útil.

3.5 Problemas que surgieron y cómo se resolvieron

Durante el desarrollo aparecieron varios problemas que no estaban previstos. La Tabla 6 los lista junto con la solución que se encontró para cada uno.

3.6 Modo de ahorro de energía

Cuando el dispositivo lleva más de 60 segundos sin detectar un dedo, entra en modo de bajo consumo: la pantalla se apaga y el sensor principal deja de trabajar. Para volver a activarlo basta con presionar un botón integrado en la carcasa, algo que cualquier niño o cuidador puede hacer de forma intuitiva, como si estuviera “despertando” al peluche. Lo interesante del modo sleep es que el código no va a ningún lado: simplemente se congela en `sleep_cpu()` y continúa exactamente en la siguiente línea cuando llega el botón.

Listing 4: Entrada y salida del modo Power-down.

```
1 // Entrar a dormir
2 set_sleep_mode(SLEEP_MODE_PWR_DOWN);
3 sleep_enable();
4 sleep_cpu(); // <-- el código se detiene aquí
5
6 // Al presionar el botón, continúa aquí
7 sleep_disable();
8 sei();
9 // reinicializar timer, sensores, LCD...
```

Cuadro 6: Problemas encontrados y soluciones aplicadas.

Problema	Solución
Bus I2C co-lapsaba con el MAX30102 conectado	Pull-up de 10kΩ en pin INT a 5 V para evitar flotación
Temperatura inválida (−206 °C) al despertar del sleep	Función <code>twi_reset()</code> antes de reinicializar sensores
Sistema no retomaba muestreo tras despertar	Reinicialización explícita del Timer1 al despertar
Lecturas RE-D/IR alternando con cero	Vaciar el FIFO completo en cada operación de lectura
SpO2 sobrestimado (99–100 %)	Recalibración a $SpO2 = 110 - 14R$
HR con resolución de saltos de 16 bpm	Cambio a cálculo por intervalo entre picos
HR inestable con variaciones extremas	Filtro de mediana sobre 7 lecturas consecutivas

3.7 Aplicación en computadora

Adicionalmente se desarrolló una pequeña aplicación en Python que recibe los datos que el dispositivo manda por cable serial y los guarda en un archivo con la fecha, hora y datos del paciente (nombre, edad, notas). Esto permite que un médico o cuidador lleve un registro de varias sesiones de medición y pueda revisarlo después. La aplicación también muestra los tres valores en tiempo real y avisa si alguno sale del rango normal.

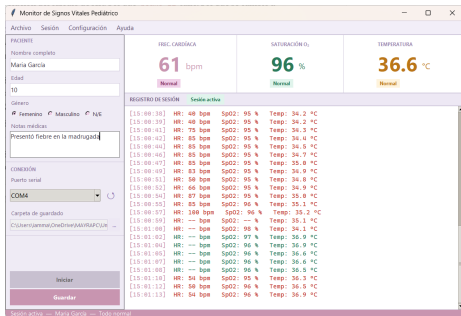


Figura 3: Captura de pantalla del software de monitoreo en PC.

3.8 Cambio en el diseño de la carcasa

Al inicio del proyecto la idea era fabricar una carcasa en impresión 3D con forma de Baymax, el personaje de *Big Hero 6*. Sin embargo, durante la fase de ensamble se decidió usar

en su lugar un peluche comercial con forma de conejita con vestido floral.

El cambio tuvo dos razones muy concretas: fabricar e imprimir la carcasa tomaba más tiempo del que había disponible, y meter la electrónica dentro de un peluche resultó mucho más práctico porque ya tiene el espacio interior listo, sin necesidad de diseñar soportes ni encajes adicionales.

En cuanto a la funcionalidad, el cambio no afecta absolutamente nada. Los sensores, la pantalla, el botón y todas las conexiones quedaron exactamente igual. Lo único diferente es el contenedor.

3.9 Cumplimiento de objetivos

El sistema final cumple con todos los objetivos planteados:

- **Frecuencia cardíaca:** precisión de ± 5 bpm tras aplicar el filtro de mediana, validada contra oxímetro comercial.
- **SpO2:** desviación de ± 2 % respecto al equipo de referencia tras la recalibración empírica.
- **Temperatura corporal:** exactitud de ± 0.3 °C respecto a termómetro digital tras la compensación de offset.
- **Visualización local:** los tres parámetros se muestran simultáneamente en la LCD 20×4 con indicador de estado.
- **Transmisión a PC:** el USART envía datos continuamente a 9600 baud en formato parseable, y la aplicación los almacena en CSV con metadatos del paciente.
- **Firmware no bloqueante:** el Timer1 controla el muestreo sin uso de `_delay_ms()` en el *loop* principal.
- **Carcasa pediátrica:** la electrónica se integra en un peluche con forma de conejita, lo cual reduce la apariencia clínica del dispositivo y favorece la aceptación del paciente pediátrico.

4. Bibliografía

Referencias

- [1] F. L. Nkoy et al., “Information needs for designing a home monitoring system for children with medical complexity,” *Int. J. Med. Inform.*, vol. 122, pp. 7–12, 2019. <https://www.sciencedirect.com/science/article/abs/pii/S1386505618308530>
- [2] E. Vonach, M. Ternek, G. Gerstweiler y H. Kaufmann, “Design of a Health Monitoring Toy for Children,” en *Proc. 15th Int. Conf. Interaction Design and Children*, 2016. <https://doi.org/10.1145/2930674.2930694>
- [3] R. Semerci, A. Kostak y S. Kuşlar, “Reducing Pain and Anxiety in the Blood Collection Unit: The Impact of the Child-Friendly Design,” *Research and Theory for Nursing Practice*, vol. 38, no. 2, pp. 252–269, 2024. <https://doi.org/10.1891/rtnp-2023-0103>
- [4] E. Prihatini et al., “Anxiety Detection for Autism Children through Vital Signs Monitoring using a Socially Assistive Robot,” *Comput. Eng. Appl.*, vol. 14, no. 1, pp. 12–21, 2025.

<https://comengapp.unsri.ac.id/index.php/comengapp/article/view/493/295>

- [5] J. N. Rojas, E. A. Manzano y M. A. Bejarano Reyes, “Remote monitoring of vital signs: pulse, temperature, and oxygen saturation using cloud computing,” *Rev. Fac. Ing.–Univ. Antioquia*, vol. 114, 2024. <https://doi.org/10.17533/udea.redin.20240941>
- [6] N. Khalafullah, N. Omran, M. Alfordogh, R. Almorabit y M. Karaim, “Designing of Portable Health Monitoring System using ATmega328P Microcontroller,” *Int. Science and Technology J.*, 2024. <https://doi.org/10.62341/namr2029>
- [7] Maxim Integrated, “MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health,” Datasheet, 2018. <https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf>
- [8] Melexis, “MLX90614 Family — Single and Dual Zone Infra Red Thermometer,” Datasheet, Rev. 12, 2019. <https://www.melexis.com/en/product/mlx90614>
- [9] Microchip Technology, “ATmega328P Datasheet,” DS40002061B, 2020. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

5. Anexos

5.1 Links

Prueba en paciente pediátrico: [video de prueba en un niño de 7 años](#)

Repositorio: [GaleyMa/Sistema-de-monitoreo-de-signos-vitales-de-uso-pediatrico](#)

5.2 Código principal del programa y código de librerías utilizadas.

El código fuente completo del proyecto se organiza en ocho archivos: `main.c` con la lógica principal, y siete módulos independientes (`twi`, `usart`, `timer1`, `max30102`, `mlx90614` y `lcd`), cada uno con su archivo de cabecera `.h` y su implementación `.c`.

A.1 `main.c`

Listing 5: `main.c` — loop principal, algoritmos y lógica de sleep.

```

1  #define F_CPU 16000000UL
2
3  #include <avr/interrupt.h>
4  #include <util/delay.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <avr/sleep.h>
8  #include <avr/power.h>
9  #include "twi.h"
10 #include "usart.h"
11 #include "mlx90614.h"
12 #include "max30102.h"
13 #include "lcd.h"
14 #include "timer1.h"
15
16 // Promedio movil temperatura
17 #define TEMP_SAMPLES 10
18 #define TEMP_OFFSET 3.2f
19 float temp_buffer[TEMP_SAMPLES];
20 uint8_t temp_index = 0;
21 uint8_t temp_filled = 0;
22
23 // Filtro de mediana para HR
24 #define HR_HISTORY 7
25 uint8_t hr_history[HR_HISTORY];
26 uint8_t hr_hist_index = 0;
27 uint8_t hr_hist_count = 0;
28
29 // Rangos normales
30 #define HR_MIN 60
31 #define HR_MAX 130
32 #define SPO2_MIN 95
33 #define TEMP_MIN 36.0f
34 #define TEMP_MAX 37.5f
35
36 volatile uint8_t just_woke_up = 0;
37
38 // Buffer IR para calculo de HR
39 #define BUFFER_SIZE 50
40 uint32_t ir_buffer[BUFFER_SIZE];
41 uint8_t buf_index = 0;
42 uint8_t buf_full = 0;
43
44 // Variables globales de medicion
45 uint8_t hr = 0;
46 uint8_t spo2 = 0;

```

```

47 uint32_t red = 0;
48 uint32_t ir = 0;
49 float temp = 0.0f;
50
51 // Contador de inactividad (~60 s a 10 Hz)
52 volatile uint16_t inactivity_counter = 0;
53 #define INACTIVITY_LIMIT 600
54
55 ISR(INT0_vect) { } // Solo despierta el micro
56
57 void setup_wakeup_interrupt(void) {
58     EICRA |= (1 << ISC01);
59     EICRA &= ~(1 << ISC00);
60     EIMSK |= (1 << INT0);
61 }
62
63 void system_sleep(void) {
64     lcd_clear();
65     lcd_set_cursor(0, 1);
66     lcd_print("___Durmiendo..._");
67     _delay_ms(1000);
68     lcd_clear();
69
70     twi_reset();
71     _delay_ms(10);
72     twi_start(LCD_ADDR << 1 | 0);
73     twi_write(0x00);
74     twi_stop();
75
76     max30102_write_reg(REG_MODE_CONFIG, 0x80);
77
78     set_sleep_mode(SLEEP_MODE_PWR_DOWN);
79     sleep_enable();
80     setup_wakeup_interrupt();
81     sleep_cpu(); // <-- el codigo se detiene aqui
82
83     sleep_disable();
84     sei();
85     just_woke_up = 1;
86 }
87
88 float smooth_temp(float new_temp) {
89     temp_buffer[temp_index++] = new_temp;
90     if (temp_index >= TEMP_SAMPLES) {
91         temp_index = 0;
92         temp_filled = 1;
93     }
94     uint8_t count = temp_filled ? TEMP_SAMPLES : temp_index;
95     float sum = 0;
96     for (uint8_t i = 0; i < count; i++) sum += temp_buffer[i];
97     return sum / count;
98 }
99
100 // SpO2 calibrado: ratio=1.0 -> 96%, ratio=1.1 -> 94.6%
101 uint8_t calculate_spo2(uint32_t red, uint32_t ir) {
102     if (ir == 0) return 0;
103     float ratio = (float)red / (float)ir;
104     int16_t val = (int16_t)(110.0f - 14.0f * ratio);
105     if (val < 0) val = 0;
106     if (val > 100) val = 100;

```

```

107     return (uint8_t)val;
108 }
109
110 uint8_t median_hr(uint8_t new_hr) {
111     hr_history[hr_hist_index++] = new_hr;
112     if (hr_hist_index >= HR_HISTORY) hr_hist_index = 0;
113     if (hr_hist_count < HR_HISTORY) hr_hist_count++;
114
115     uint8_t sorted[HR_HISTORY];
116     for (uint8_t i = 0; i < hr_hist_count; i++)
117         sorted[i] = hr_history[i];
118
119     for (uint8_t i = 0; i < hr_hist_count - 1; i++)
120         for (uint8_t j = 0; j < hr_hist_count - i - 1; j++)
121             if (sorted[j] > sorted[j+1]) {
122                 uint8_t tmp = sorted[j];
123                 sorted[j] = sorted[j+1];
124                 sorted[j+1] = tmp;
125             }
126
127     return sorted[hr_hist_count / 2];
128 }
129
130 // HR por intervalo promedio entre picos
131 uint8_t calculate_hr(uint32_t *buf, uint8_t size,
132                     uint8_t *hr_out) {
133     uint32_t max_val = buf[0], min_val = buf[0];
134     for (uint8_t i = 1; i < size; i++) {
135         if (buf[i] > max_val) max_val = buf[i];
136         if (buf[i] < min_val) min_val = buf[i];
137     }
138
139     uint32_t amplitude = max_val - min_val;
140     if (amplitude < 500) { *hr_out = 0; return 0; }
141
142     uint32_t threshold = min_val + (amplitude * 6 / 10);
143
144     uint8_t peak_positions[15];
145     uint8_t num_peaks = 0;
146     uint8_t was_below = (buf[0] < threshold) ? 1 : 0;
147     uint8_t last_peak = 0;
148
149     for (uint8_t i = 1; i < size; i++) {
150         if (was_below && buf[i] >= threshold
151             && (i - last_peak) >= 3) {
152             if (num_peaks < 15)
153                 peak_positions[num_peaks++] = i;
154             was_below = 0;
155             last_peak = i;
156         } else if (buf[i] < threshold) {
157             was_below = 1;
158         }
159     }
160
161     if (num_peaks < 2) { *hr_out = 0; return 0; }
162
163     uint16_t total = 0;
164     for (uint8_t i = 1; i < num_peaks; i++)
165         total += peak_positions[i] - peak_positions[i-1];
166     float avg = (float)total / (num_peaks - 1);

```

```

167
168     *hr_out = (uint8_t)(600.0f / avg);
169     return (*hr_out >= 40 && *hr_out <= 180);
170 }
171
172 void update_lcd(uint8_t hr, uint8_t spo2, float temp) {
173     char buf[8];
174
175     lcd_set_cursor(0, 0);
176     lcd_print("HR:_____");
177     if (hr > 0) { itoa(hr, buf, 10); lcd_print(buf);
178                 lcd_print("_bpm"); }
179     else         lcd_print("--_bpm");
180
181     lcd_set_cursor(0, 1);
182     lcd_print("SpO2:_____");
183     if (spo2 > 0) { itoa(spo2, buf, 10); lcd_print(buf);
184                 lcd_print("_%"); }
185     else         lcd_print("--_%");
186
187     lcd_set_cursor(0, 2);
188     lcd_print("Temp:___");
189     dtostrf(temp, 5, 1, buf);
190     lcd_print(buf);
191     lcd_print("_C___");
192
193     lcd_set_cursor(0, 3);
194     lcd_print("_____");
195 }
196
197 int main(void) {
198     usart_init();
199     twi_init();
200     timer1_init();
201     lcd_init();
202
203     // Verificar sensor
204     if (max30102_read_reg(REG_PART_ID) != 0x15) {
205         lcd_set_cursor(0, 0);
206         lcd_print("ERROR:_MAX30102");
207         usart_send_string("ERROR:_MAX30102_no_encontrado\r\n");
208         while (1);
209     }
210
211     max30102_write_reg(REG_FIFO_CONFIG, 0x10);
212     max30102_init();
213
214     lcd_clear();
215     lcd_set_cursor(0, 0); lcd_print("___Sistema_Listo___");
216     lcd_set_cursor(0, 1); lcd_print("___Pon_el_dedo___");
217     lcd_set_cursor(0, 2); lcd_print("___en_el_sensor___");
218     _delay_ms(2000);
219     lcd_clear();
220
221     while (1) {
222         loop_start:
223
224         if (just_woke_up) {
225             just_woke_up = 0;
226             twi_reset();

```

```

227     _delay_ms(100);
228     timer1_init();
229     max30102_write_reg(REG_FIFO_CONFIG, 0x10);
230     max30102_init();
231     lcd_init();
232     inactivity_counter = 0;
233     buf_index = 0; buf_full = 0;
234     hr = 0; spo2 = 0;
235     temp_index = 0; temp_filled = 0;
236     hr_hist_index = 0; hr_hist_count = 0;
237     usart_send_string("Sistema_despertado\r\n");
238 }
239
240 if (inactivity_counter >= INACTIVITY_LIMIT) {
241     system_sleep();
242     goto loop_start;
243 }
244
245 if (sample_flag) {
246     sample_flag = 0;
247     inactivity_counter++;
248
249     if (ir > 5000) inactivity_counter = 0;
250
251     temp = smooth_temp(
252         mlx90614_read_temp_object() + TEMP_OFFSET);
253
254     if (max30102_read_fifo(&red, &ir)) {
255         if (ir < 5000) {
256             hr = 0; spo2 = 0;
257             buf_index = 0; buf_full = 0;
258             hr_hist_index = 0; hr_hist_count = 0;
259             max30102_reset_fifo();
260         } else if (red < 200000 && ir < 200000) {
261             ir_buffer[buf_index++] = ir;
262             if (buf_index >= BUFFER_SIZE) {
263                 buf_index = 0; buf_full = 1;
264             }
265             spo2 = calculate_spo2(red, ir);
266             if (buf_full) {
267                 uint8_t hr_calc;
268                 if (calculate_hr(ir_buffer,
269                     BUFFER_SIZE, &hr_calc))
270                     hr = median_hr(hr_calc);
271             }
272         } else {
273             max30102_reset_fifo();
274         }
275     }
276
277     update_lcd(hr, spo2, temp);
278
279     char buf[8];
280     usart_send_string("HR:");
281     itoa(hr, buf, 10); usart_send_string(buf);
282     usart_send_string("_SpO2:");
283     itoa(spo2, buf, 10); usart_send_string(buf);
284     usart_send_string("_Temp:");
285     dtostrf(temp, 5, 1, buf); usart_send_string(buf);
286     usart_send_string("\r\n");

```

```

287     }
288 }
289 }

```

A.2 twi.h/twi.c

Listing 6: twi.h — interfaz del driver I2C.

```

1  #ifndef TWI_H
2  #define TWI_H
3
4  #include <avr/io.h>
5
6  #define F_SCL      100000UL
7  #define F_CPU_HZ   16000000UL
8  #define TWBR_VAL   ((F_CPU_HZ / F_SCL - 16) / 2)
9
10 #define TWI_START      0x08
11 #define TWI_REP_START   0x10
12 #define TWI_MT_SLA_ACK  0x18
13 #define TWI_MT_DATA_ACK 0x28
14 #define TWI_MR_SLA_ACK  0x40
15 #define TWI_MR_DATA_ACK 0x50
16 #define TWI_MR_DATA_NACK 0x58
17
18 void twi_init(void);
19 uint8_t twi_start(uint8_t address);
20 void twi_stop(void);
21 uint8_t twi_write(uint8_t data);
22 uint8_t twi_read_ack(void);
23 uint8_t twi_read_nack(void);
24 void twi_reset(void);
25
26 #endif

```

Listing 7: twi.c — implementación del driver I2C bare-metal.

```

1  #include "twi.h"
2  #include <util/delay.h>
3
4  void twi_init(void) {
5      TWSR = 0x00;
6      TWBR = TWBR_VAL;
7      TWCR = (1 << TWEN);
8  }
9
10 static uint8_t twi_wait(void) {
11     uint16_t timeout = 10000;
12     while (!(TWCR & (1 << TWINT)))
13         if (--timeout == 0) { twi_stop(); return 1; }
14     return 0;
15 }
16
17 uint8_t twi_start(uint8_t address) {
18     uint16_t timeout;
19     TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
20     timeout = 10000;
21     while (!(TWCR & (1 << TWINT)))
22         if (--timeout == 0) { twi_stop(); return 1; }
23

```

```

24     uint8_t status = TWSR & 0xF8;
25     if (status != TWI_START && status != TWI_REP_START)
26         return 1;
27
28     TWDR = address;
29     TWCR = (1 << TWINT) | (1 << TWEN);
30     timeout = 10000;
31     while (!(TWCR & (1 << TWINT)))
32         if (--timeout == 0) { twi_stop(); return 1; }
33
34     status = TWSR & 0xF8;
35     if (status != TWI_MT_SLA_ACK && status != TWI_MR_SLA_ACK)
36         return 1;
37     return 0;
38 }
39
40 void twi_stop(void) {
41     TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN);
42 }
43
44 uint8_t twi_write(uint8_t data) {
45     TWDR = data;
46     TWCR = (1 << TWINT) | (1 << TWEN);
47     if (twi_wait()) return 1;
48     if ((TWSR & 0xF8) != TWI_MT_DATA_ACK) return 1;
49     return 0;
50 }
51
52 uint8_t twi_read_ack(void) {
53     TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWEA);
54     twi_wait();
55     return TWDR;
56 }
57
58 uint8_t twi_read_nack(void) {
59     TWCR = (1 << TWINT) | (1 << TWEN);
60     twi_wait();
61     return TWDR;
62 }
63
64 void twi_reset(void) {
65     TWCR = 0;
66     _delay_ms(10);
67     TWSR = 0x00;
68     TWBR = TWBR_VAL;
69     TWCR = (1 << TWEN);
70     _delay_ms(10);
71 }

```

A.3 usart.h/usart.c

Listing 8: usart.h — interfaz UART.

```

1  #ifndef USART_H
2  #define USART_H
3
4  #include <avr/io.h>
5
6  #define BAUD      9600
7  #define UBRR_VAL (F_CPU / 16 / BAUD - 1)

```

```

8
9 void usart_init(void);
10 void usart_send_char(char c);
11 void usart_send_string(const char *str);
12 void usart_send_float(float val, uint8_t decimals);
13
14 #endif

```

Listing 9: usart.c — implementación UART 9600 baud.

```

1 #include "usart.h"
2 #include <stdlib.h>
3
4 void usart_init(void) {
5     UBRR0H = (UBRR_VAL >> 8);
6     UBRR0L = UBRR_VAL;
7     UCSR0B = (1 << TXEN0) | (1 << RXEN0);
8     UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
9 }
10
11 void usart_send_char(char c) {
12     while (!(UCSR0A & (1 << UDRE0)));
13     UDR0 = c;
14 }
15
16 void usart_send_string(const char *str) {
17     while (*str) usart_send_char(*str++);
18 }
19
20 void usart_send_float(float val, uint8_t decimals) {
21     char buf[16];
22     dtostrf(val, 6, decimals, buf);
23     usart_send_string(buf);
24 }

```

A.4 timer1.h/timer1.c

Listing 10: timer1.h — interfaz del Timer1 CTC.

```

1 #ifndef TIMER1_H
2 #define TIMER1_H
3
4 #include <avr/io.h>
5 #include <avr/interrupt.h>
6
7 extern volatile uint8_t sample_flag;
8
9 void timer1_init(void);
10
11 #endif

```

Listing 11: timer1.c — Timer1 CTC a 10 Hz (interrupcin cada 100 ms).

```

1 #include "timer1.h"
2
3 volatile uint8_t sample_flag = 0;
4
5 void timer1_init(void) {
6     TCCR1A = 0;
7     TCCR1B = 0; // Detener timer primero

```

```

8   TCNT1 = 0;           // Reset contador
9   OCR1A = 6249;        // 16MHz / (256 * (6249+1)) = 10 Hz
10  TCCR1B = (1 << WGM12) | (1 << CS12); // CTC + prescaler 256
11  TIMSK1 = (1 << OCIE1A);
12  sei();
13 }
14
15 ISR(TIMER1_COMPA_vect) {
16     sample_flag = 1;
17 }

```

A.5 max30102.h/max30102.c

Listing 12: max30102.h — interfaz del sensor MAX30102.

```

1  #ifndef MAX30102_H
2  #define MAX30102_H
3
4  #include <avr/io.h>
5
6  #define MAX30102_ADDR 0x57
7
8  #define REG_INT_STATUS1 0x00
9  #define REG_INT_STATUS2 0x01
10 #define REG_INT_ENABLE1 0x02
11 #define REG_INT_ENABLE2 0x03
12 #define REG_FIFO_WR_PTR 0x04
13 #define REG_OVF_COUNTER 0x05
14 #define REG_FIFO_RD_PTR 0x06
15 #define REG_FIFO_DATA 0x07
16 #define REG_FIFO_CONFIG 0x08
17 #define REG_MODE_CONFIG 0x09
18 #define REG_SPO2_CONFIG 0x0A
19 #define REG_LED1_PA 0x0C
20 #define REG_LED2_PA 0x0D
21 #define REG_PART_ID 0xFF
22
23 void max30102_init(void);
24 uint8_t max30102_read_reg(uint8_t reg);
25 void max30102_write_reg(uint8_t reg, uint8_t val);
26 uint8_t max30102_read_fifo(uint32_t *red, uint32_t *ir);
27 void max30102_reset_fifo(void);
28
29 #endif

```

Listing 13: max30102.c — inicialización y lectura del FIFO.

```

1  #include "max30102.h"
2  #include "twi.h"
3  #include <util/delay.h>
4
5  void max30102_write_reg(uint8_t reg, uint8_t val) {
6      twi_start(MAX30102_ADDR << 1 | 0);
7      twi_write(reg);
8      twi_write(val);
9      twi_stop();
10 }
11
12 uint8_t max30102_read_reg(uint8_t reg) {
13     twi_start(MAX30102_ADDR << 1 | 0);

```

```

14     twi_write(reg);
15     twi_start(MAX30102_ADDR << 1 | 1);
16     uint8_t val = twi_read_nack();
17     twi_stop();
18     return val;
19 }
20
21 void max30102_init(void) {
22     max30102_write_reg(REG_MODE_CONFIG, 0x40); // Reset
23     _delay_ms(100);
24     max30102_write_reg(REG_FIFO_CONFIG, 0x10); // Rollover on
25     max30102_write_reg(REG_MODE_CONFIG, 0x03); // Modo SpO2
26     max30102_write_reg(REG_SPO2_CONFIG, 0x27); // 400Hz, 411us
27     max30102_write_reg(REG_LED1_PA, 0x0C); // LED rojo ~2.4mA
28     max30102_write_reg(REG_LED2_PA, 0x0C); // LED IR ~2.4mA
29     max30102_write_reg(REG_FIFO_WR_PTR, 0x00);
30     max30102_write_reg(REG_OVF_COUNTER, 0x00);
31     max30102_write_reg(REG_FIFO_RD_PTR, 0x00);
32 }
33
34 // Lee y vacia el FIFO; devuelve la ultima muestra valida
35 uint8_t max30102_read_fifo(uint32_t *red, uint32_t *ir) {
36     uint8_t wr = max30102_read_reg(REG_FIFO_WR_PTR);
37     uint8_t rd = max30102_read_reg(REG_FIFO_RD_PTR);
38     if (wr == rd) return 0;
39
40     uint32_t last_ir = 0, last_red = 0;
41     uint8_t samples = 0;
42
43     while (wr != rd) {
44         twi_start(MAX30102_ADDR << 1 | 0);
45         twi_write(REG_FIFO_DATA);
46         twi_start(MAX30102_ADDR << 1 | 1);
47         uint8_t b0 = twi_read_ack();
48         uint8_t b1 = twi_read_ack();
49         uint8_t b2 = twi_read_ack();
50         uint8_t b3 = twi_read_ack();
51         uint8_t b4 = twi_read_ack();
52         uint8_t b5 = twi_read_nack();
53         twi_stop();
54
55         // Nota: clon MH-ET LIVE tiene canales invertidos
56         last_ir = ((uint32_t)(b0 & 0x03) << 16)
57             | ((uint32_t)b1 << 8) | b2;
58         last_red = ((uint32_t)(b3 & 0x03) << 16)
59             | ((uint32_t)b4 << 8) | b5;
60
61         rd = (rd + 1) & 0x1F;
62         max30102_write_reg(REG_FIFO_RD_PTR, rd);
63         samples++;
64         wr = max30102_read_reg(REG_FIFO_WR_PTR);
65     }
66
67     if (samples == 0) return 0;
68     *ir = last_ir;
69     *red = last_red;
70     return 1;
71 }
72
73 void max30102_reset_fifo(void) {

```

```

74     max30102_write_reg(REG_FIFO_WR_PTR, 0x00);
75     max30102_write_reg(REG_OVF_COUNTER, 0x00);
76     max30102_write_reg(REG_FIFO_RD_PTR, 0x00);
77 }

```

A.6 mlx90614.h/mlx90614.c

Listing 14: mlx90614.h — interfaz del termómetro IR.

```

1  #ifndef MLX90614_H
2  #define MLX90614_H
3
4  #include <avr/io.h>
5
6  #define MLX90614_ADDR 0x5A
7
8  float mlx90614_read_temp_object(void);
9  float mlx90614_read_temp_ambient(void);
10
11 #endif

```

Listing 15: mlx90614.c — lectura SMBus del sensor de temperatura IR.

```

1  #include "mlx90614.h"
2  #include "twi.h"
3
4  static float mlx90614_read(uint8_t reg) {
5      uint8_t lsb, msb;
6      twi_start(MLX90614_ADDR << 1 | 0);
7      twi_write(reg);
8      twi_start(MLX90614_ADDR << 1 | 1);
9      lsb = twi_read_ack();
10     msb = twi_read_ack();
11     twi_read_nack(); // PEC (ignorado)
12     twi_stop();
13
14     uint16_t raw = ((uint16_t)msb << 8) | lsb;
15     return (raw * 0.02f) - 273.15f; // Kelvin -> Celsius
16 }
17
18 float mlx90614_read_temp_object(void) { return mlx90614_read(0x07); }
19 float mlx90614_read_temp_ambient(void) { return mlx90614_read(0x06); }

```

A.7 lcd.h/lcd.c

Listing 16: lcd.h — interfaz del driver LCD HD44780 via PCF8574.

```

1  #ifndef LCD_H
2  #define LCD_H
3
4  #include <avr/io.h>
5
6  #define LCD_ADDR 0x27
7  #define LCD_COLS 20
8  #define LCD_ROWS 4
9
10 // Bits del PCF8574
11 #define LCD_RS (1 << 0) // Register Select
12 #define LCD_RW (1 << 1) // Read/Write
13 #define LCD_EN (1 << 2) // Enable

```

```

14 #define LCD_BL (1 << 3) // Backlight
15
16 void lcd_init(void);
17 void lcd_clear(void);
18 void lcd_set_cursor(uint8_t col, uint8_t row);
19 void lcd_print(const char *str);
20 void lcd_print_char(char c);
21 void lcd_create_char(uint8_t location, uint8_t *charmap);
22
23 #endif

```

Listing 17: lcd.c — driver HD44780 en modo 4 bits a través del PCF8574.

```

1 #include "lcd.h"
2 #include "twi.h"
3 #include <util/delay.h>
4
5 static uint8_t backlight = LCD_BL;
6
7 static void pcf8574_write(uint8_t data) {
8     twi_start(LCD_ADDR << 1 | 0);
9     twi_write(data | backlight);
10    twi_stop();
11 }
12
13 static void lcd_pulse_enable(uint8_t data) {
14     pcf8574_write(data | LCD_EN); _delay_us(1);
15     pcf8574_write(data & ~LCD_EN); _delay_us(50);
16 }
17
18 static void lcd_write_nibble(uint8_t nibble, uint8_t rs) {
19     uint8_t data = (nibble << 4) | rs;
20     pcf8574_write(data);
21     lcd_pulse_enable(data);
22 }
23
24 static void lcd_write_byte(uint8_t byte, uint8_t rs) {
25     lcd_write_nibble(byte >> 4, rs);
26     lcd_write_nibble(byte & 0x0F, rs);
27     _delay_us(100);
28 }
29
30 static void lcd_command(uint8_t cmd) {
31     lcd_write_byte(cmd, 0);
32 }
33
34 void lcd_print_char(char c) {
35     lcd_write_byte(c, LCD_RS);
36 }
37
38 void lcd_init(void) {
39     _delay_ms(50);
40     // Secuencia de inicializacion HD44780 en modo 4 bits
41     lcd_write_nibble(0x03, 0); _delay_ms(5);
42     lcd_write_nibble(0x03, 0); _delay_us(150);
43     lcd_write_nibble(0x03, 0); _delay_us(150);
44     lcd_write_nibble(0x02, 0); // Entrar en modo 4 bits
45
46     lcd_command(0x28); // 4 bits, 2 lineas, 5x8 dots
47     lcd_command(0x0C); // Display ON, cursor OFF

```

```
48     lcd_command(0x06); // Incrementar cursor automaticamente
49     lcd_command(0x01); // Limpiar pantalla
50     _delay_ms(2);
51 }
52
53 void lcd_clear(void) {
54     lcd_command(0x01);
55     _delay_ms(2);
56 }
57
58 void lcd_set_cursor(uint8_t col, uint8_t row) {
59     uint8_t offsets[] = {0x00, 0x40, 0x14, 0x54};
60     lcd_command(0x80 | (col + offsets[row]));
61 }
62
63 void lcd_print(const char *str) {
64     while (*str) lcd_print_char(*str++);
65 }
66
67 void lcd_create_char(uint8_t location, uint8_t *charmap) {
68     lcd_command(0x40 | ((location & 0x07) << 3));
69     for (uint8_t i = 0; i < 8; i++)
70         lcd_print_char(charmap[i]);
71 }
```